1. Core Pointer Concepts & Terminology

The Address-of Operator (&)

Definition: A unary operator used to get the memory location (address) of a variable.

Characteristics:

We **cannot change** or decide the memory address of a variable; it is allocated automatically by the system.

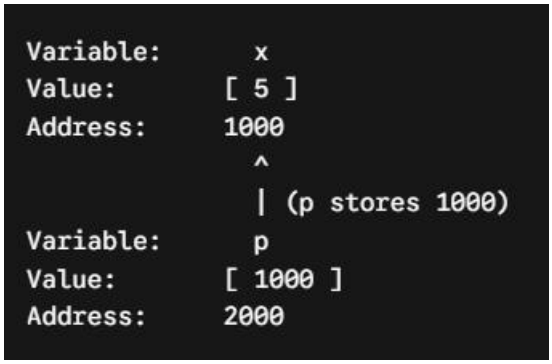
&x acts as an r-value (constant reference value). You cannot do &x = 7; because assigning a value to a constant expression results in a compilation error.

The Indirection / Dereferencing Operator (*)

Definition: A unary operator used to access or modify the value stored at a specific memory location.

Notation: *variable_name reads or writes directly to the memory address the pointer holds.

Visualizing Basic Pointers :



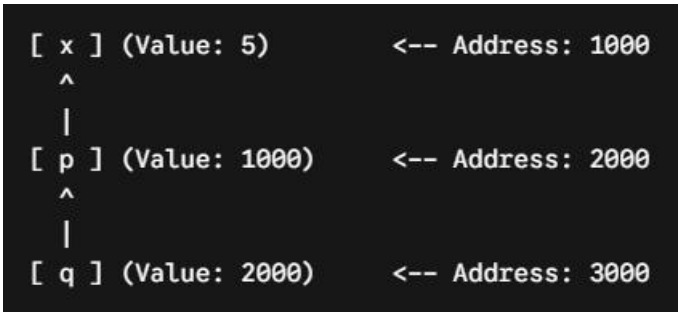
Code snippet verification:

```
int x = 5;
int *j = &x;    ----> Assuming &x = 1000, &j = 2000
printf("%d %d %d", j, &x, x);    ----> Output: 1000 1000 5
printf("%d %d %d", *j, *&x, &j);  -----> Output: 5 5 2000
```

2. Pointer-to-Pointer (Multiple Indirection)

When a pointer stores the memory location of another pointer rather than an ordinary variable, we use a pointer-to-pointer ().

Memory Diagram



int x = 5;
int *p = &x; **-----> Stores address of ordinary variable x**
int **q = &p; **-----> Stores address of pointer variable p**

3. Pointer Arithmetic Rules

Invalid Operations

You **cannot** perform the following operations with two addresses:

p + q (Addition of two addresses)

p * q (Multiplication)

p / q (Division)

Valid Operations

Pointer + Integer: Moves the pointer forward in memory.

New Address = Current Address + Integer * sizeof (data_type)

Pointer - Integer: Moves the pointer backward in memory.

New Address = Current Address - Integer * sizeof (data_type)

Pointer - Pointer: Finds the number of element positions separating the two pointers (both must point to the same array).

$$\text{Elements Separating} = \frac{\text{Address}_2 - \text{Address}_1}{\text{sizeof(data\_type)}}$$

4. Parameter Passing: Value vs. Reference

Method	Definition	Effect on Original Value	Diagram
Call by Value	Passes a direct copy of the data.	**No effect** on the original variable.	Main(x) -> Copy(x') inside Function
Call by Reference	Passes the memory address of the data using pointers.	**Modifies** the original variable directly.	Function(*p) -> Accesses/Alters Main(x)

5. Pointers and Arrays

In C, an array name decays into a pointer to its first element.

Notation Equivalence: a[i] is identical to *(a + i)

Multi-Dimensional Array Deconstruction

2D Array: a[3][2] transforms structurally to *(*(a + 3) + 2)

3D Array: a[3][5][2] transforms structurally to *(*(*(a + 3) + 5) + 2)

Array of Pointers

An array of pointers stores memory addresses instead of ordinary data values.

```
#include <stdio.h>
int main()
{
   int a[] = {11, 22, 33, 44};
   int *ptr[] = {a, a + 1, a + 2, a + 3};  -----> Stores element addresses
   int i = 1;

   while(i < 4)
   {
      printf("%d\t", *(*(ptr + (i - 1))));
      i++;
   }
   return 0;
}
```

Output: 11 22 33

6. Special Pointer Classifications

Void Pointer (void *)

Definition: A generic pointer that can store the address of any data type without an associative type declaration.

Rule: You **cannot** directly dereference a void pointer without **type-casting** it first, because the compiler does not know how many bytes to read.

```
void *vp;
int a = 5;
vp = &a;
printf("%d", *vp);  ----> ERROR
printf("%d", *(int *)vp);  -----> CORRECT
```

Outputs: 5

Null Pointer (NULL)

Definition: A pointer explicitly pointed to 0 or NULL to signify it points to nothing valid.

Rule: Dereferencing a NULL pointer causes a program crash.

Safety Benefit: ptr1 == ptr2 is always guaranteed true if both are initialized to NULL. Uninitialized pointers provide no such guarantee.

Wild Pointer

Definition: An **uninitialized pointer** that holds random garbage memory addresses.

Risk: Dereferencing it causes unpredictable undefined behavior or memory bugs.

Dangling Pointer

Definition: A pointer that points to a memory location that has already been deallocated/freed.

Step 1: Allocation [ptr] --------> [Memory Allocated (10)]
Step 2: free(ptr) [ptr] --------> [Deallocated Area (Garbage)] (Dangling!)
Step 3: Reset [ptr] --------> NULL (Safe!)

💡 **Best Practice Remedy:** To eliminate bugs caused by both Wild and Dangling pointers, always initialize unassigned pointers to NULL, and set pointers to NULL immediately after calling free().

📝 **Note:** For making your understanding more clear and simple, you must visit the handwritten note provided in the same folder **12-Pointer**. It contains matching variable examples, diagrams, and step-by-step walkthroughs to reinforce these mechanics.